

DOKUMENTACJA TECHNICZNA

Aplikacja implementująca algorytm ECDSA do podpisywania wiadomości i weryfikacji podpisów

Aleksander Brachman, Kamil Maciaszek, Krzysztof Skociński

1. Wstęp

Niniejsza aplikacja implementuje *Elliptic Curve Digital Signature Algorithm* (ECDSA), który jest częścią kryptografii opartej na krzywych eliptycznych. ECDSA umożliwia generowanie bezpiecznego podpisu cyfrowego wiadomości, pliku, czy oprogramowania przy wykorzystaniu krótszych kluczy (w bitach), aniżeli przy użyciu algorytmu RSA. ECDSA definiuje również kroki konieczne do zweryfikowania poprawności podpisu cyfrowego, który wcześniej został wygenerowany przez ten algorytm.

W ramach aplikacji zaprogramowano różne operacje matematyczne, będące podstawą działania ECDSA, a które, złożone w całość, umożliwiają użytkownikowi aplikacji podpisanie wpisanej przez siebie wiadomości, z wykorzystaniem wybranej krzywej eliptycznej. Dodatkową funkcjonalnością aplikacji jest możliwość weryfikacji prawidłowości wygenerowanego wcześniej podpisu cyfrowego ECDSA dla danej wiadomości.

2. Wymagania techniczne

Kod aplikacji udostępniony jest w poniższym linku:

https://colab.research.google.com/drive/1ejEHBW-zGegK1m_o5msvTpiDvnJtPYS4?usp=sharing

Zalecany środowiskiem uruchomienia aplikacji jest Google Colab. Aby uruchomić poszczególne komórki z kodem aplikacji należy zalogować się na swoje konto Google.

Uwaga: W przypadku chęci pobrania pliku .ipynb z kodem i pracy z aplikacją w trybie offline, konieczne jest posiadanie zainstalowanego środowiska Python w swoim systemie operacyjnym oraz doinstalowanie wymaganych bibliotek takich jak secrets, hashlib, czy ecdsa.

3. Opis aplikacji

Kod aplikacji składa się z kilku komórek kodu języka Python; każda komórka zawiera konieczne do prawidłowego działania aplikacji linijki kodu.

```
[ ] import secrets
    from hashlib import sha256
    import time
    from ecdsa import SigningKey, NIST224p, NIST256p, NIST384p, NIST521p

[ ] !pip install ecdsa
```

Na początku aplikacja musi zaimportować wszelkie potrzebne biblioteki, które wykorzystywane są w implementacji. Są to biblioteki:

- *secrets*, która umożliwia generowanie *kryptograficznie silnych* liczb losowych,
- *hashlib*, a dokładniej metoda *sha256*, która generuje skrót (hash) wiadomości,

- *time*, która wykorzystywana jest do mierzenia czasu procesów zachodzących w aplikacji,
- *ecdsa*, która dostarcza w języku Python gotowe metody, umożliwiające korzystanie z ECDSA. Została zaimportowana celem porównania „ręcznej” implementacji ECDSA z „gotowym” rozwiązaniem.

W przypadku problemów z importem którejs z bibliotek w Google Colab dostępna jest komórka z poleceniem `!pip install <nazwa_biblioteki>`, która instaluje w używanym środowisku Python brakującą bibliotekę.

Następnie zdefiniowana została komórka zawierająca funkcje główne implementujące algorytm ECDSA w aplikacji.

```
[ ] # Funkcje główne

# Funkcja umożliwiająca wybór krzywej
def choose_curve():
    while True:
        print("Wybierz krzywą eliptyczną na jakiej chcesz dokonać podpisu:")
        print("1 - P-224, 2 - P-256, 3 - P-384, 4 - P-521")
        chosen_curve = input("Podaj cyfrę: ").strip()
        if chosen_curve == "1":
            # Parametry krzywej P-224
            p = 26959946667150639794667015087019630673557916260026308143510066298881 # modulo krzywej - duża liczba pierwsza
            a = -3 # współczynnik krzywej
            b = 18958286285566608000408668544493926415504680968679321075787234672564 # współczynnik krzywej
            Gx = 19277929113566293071110308034699488026831934219452440156649784352033 # współrzędna x generatora
            Gy = 19926808758034470970197974370888749184205991990603949537637343198772 # współrzędna y generatora
            n = 26959946667150639794667015087019625940457807714424391721682722368061 # rząd generatora
            cur = "P-224" # nazwa krzywej
            G = (Gx, Gy) # tworzenie punktu generatora o współrzędnych x i y
            print("Wybrałeś krzywą P-224.")
            return p, a, b, G, n, cur
```

Pierwszą funkcją w tej komórce jest *choose_curve()*. We funkcji zdefiniowane zostały parametry krzywych eliptycznych dostępnych dla użytkownika w aplikacji. Są to krzywe standaryzowane przez NIST i przeznaczone do użytku w ECDSA: P-224, P-256, P-384 oraz P-521. Liczba w nazwie określa długość klucza w bitach.

```
# Funkcja generująca klucz prywatny i publiczny
def generate_keys():
    d = secrets.randbelow(n - 1) + 1 # Klucz prywatny, losowa liczba z zakresu [1, n-1], gdzie n to rząd G
    Q = point_mul(d, G) # Klucz publiczny Q = d * G, Q również znajduje się na krzywej
    return d, Q
```

Na podstawie parametrów *n* i *G* z funkcji *choose_curve()* generowana jest w funkcji *generate_keys()*, zgodnie z ECDSA, para kluczy: klucz prywatny oraz klucz publiczny, które są później wykorzystywane do podpisu wiadomości i weryfikacji podpisu.

```
# Funkcja tworzenia podpisu
def sign_message(message, d, G):
    z = int(sha256(message.encode()).hexdigest(), 16) # Hash wiadomości
    while True:
        k = secrets.randbelow(n - 1) + 1 # Losowa liczba z zakresu [1, n-1], gdzie n to rząd punktu generatora G
        P = point_mul(k, G) # P = k * G
        r = P[0] % n # następuje redukcja modulo współrzędnej x punktu P = r to współrzędna x modulo n
        if r == 0:
            continue # Powtarzamy proces, jeśli r = 0 (czyli gdy współrzędna x punktu P była podzielna przez n)
        s = (mod_inv(k, n) * (z + r * d)) % n
        if s != 0:
            break # Powtórz, jeśli s = 0
    return (r, s) # obie wartości będą konieczne do późniejszej weryfikacji podpisu wiadomości
```

Funkcja *sign_message()* dokonuje podpisu wiadomości, zgodnie z operacjami matematycznymi zdefiniowanymi w ECDSA – wykorzystywany jest m.in. wygenerowany wcześniej klucz prywatny.

```

# Weryfikacja podpisu
def verify_signature(public_key, message, signature):
    r, s = signature
    if not (1 <= r < n and 1 <= s < n):
        return False

    z = int(sha256(message.encode()).hexdigest(), 16) # Hash wiadomości

    w = mod_inv(s, n)

    u1 = z * w % n
    u2 = r * w % n

    u1G = point_mul(u1, G)
    u2Q = point_mul(u2, public_key)
    x, y = point_add(u1G, u2Q)

    # Sprawdzenie czy r == x mod N - jeżeli tak: podpis jest poprawny
    if x % n == r:
        return True
    return False

```

Funkcja *verify_signature()* odpowiada za weryfikację poprawności podpisu bazując na wygenerowanym wcześniej kluczu publicznym, wiadomości, dla której wygenerowano podpis oraz samym podpisie.

Bardzo ważną częścią aplikacji są również tzw. funkcje pomocnicze, które umożliwiają wykonywanie skomplikowanych operacji matematycznych, będących częścią ECDSA – odwrotność modularna liczby a modulo p , czy mnożenie skalarne punktu P przez skalar k .

```

# Funkcje pomocnicze

# Odwrotność modularna a mod p
def mod_inv(a, p):
    return pow(a, -1, p)

# Dodawanie dwóch punktów na krzywej eliptycznej
def point_add(P, Q):
    if P == (None, None):
        return Q
    if Q == (None, None):
        return P

    x1, y1 = P
    x2, y2 = Q

    if x1 == x2 and y1 != y2:
        return (None, None) # Punkt w nieskończoności
    if P == Q:
        # Punkt podwojenia
        m = (3 * x1**2 + a) * mod_inv(2 * y1, p) % p
    else:
        # Dodawanie dwóch różnych punktów
        m = (y2 - y1) * mod_inv(x2 - x1, p) % p

    x3 = (m**2 - x1 - x2) % p
    y3 = (m * (x1 - x3) - y1) % p

    return (x3, y3)

```

```

# Mnożenie skalarne punktu P przez skalar k
def point_mul(k, P):
    Q = (None, None) # Punkt w nieskończoności
    while k:
        if k & 1:
            Q = point_add(Q, P)
            P = point_add(P, P)
            k >>= 1
    return Q

```

```

# Funkcja umożliwiająca załadowanie tekstu jako wiadomości z pliku .txt
def load_text_file(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            text_content = file.read()
        return text_content
    except Exception as e:
        print(f"Wystąpił błąd podczas otwierania pliku: {e}")
    return None

# Przykład użycia
file_path = "example.txt" # Podaj ścieżkę do pliku
message = load_text_file(file_path)

```

W komórce z funkcjami pomocniczymi można również odnaleźć domyślnie „zakomentowaną” funkcję `load_text_file()`, która umożliwia pobranie tekstu z pliku `.txt` jako wiadomości, dla której następnie wygenerowany zostaje podpis cyfrowy.

Aplikacja zawiera również komórkę z funkcjami opierającymi się na metodach biblioteki `ecdsa`. Jak zostało wspomniane na początku dokumentacji, funkcje te zostały zaimplementowane aby porównać wydajność „własnej” implementacji ECDSA z gotowym rozwiązaniem, którym jest ta biblioteka.

```
# Gotowa implementacja - biblioteka ecdsa

# Tworzenie podpisu
def python_ecdsa_module_signature(cur, message):
    if cur == "P-224":
        curve = NIST224p
    elif cur == "P-256":
        curve = NIST256p
    elif cur == "P-384":
        curve = NIST384p
    elif cur == "P-521":
        curve = NIST521p

    sk = SigningKey.generate(curve)
    vk = sk.verifying_key
    signature = sk.sign(message.encode())
    return signature, vk
```

Funkcja `python_ecdsa_module_signature()` generuje podpis cyfrowy dla danej krzywej eliptycznej oraz wiadomości.

```
# Weryfikowanie podpisu
def python_ecdsa_module_verify(cur, signature, message, vk):
    if cur == "P-224":
        curve = NIST224p
    elif cur == "P-256":
        curve = NIST256p
    elif cur == "P-384":
        curve = NIST384p
    elif cur == "P-521":
        curve = NIST521p

    verify = vk.verify(signature, message.encode())
    return verify
```

Funkcja `python_ecdsa_module_verify()` weryfikuje prawidłowość podpisu cyfrowego dla danej krzywej, podpisu oraz wiadomości.

Uruchomienie głównej komórki aplikacji uruchamia całą aplikację.

Uwaga: Aby uniknąć błędów, przed uruchomieniem głównej komórki konieczne jest uruchomienie wszystkich poprzednich komórek z kodem!

```
# Wybór krzywej
p, a, b, G, n, cur = choose_curve()

# Generowanie kluczy
start_time1 = time.time()
d, Q = generate_keys()
end_time1 = time.time()
print("Czas generowania kluczy: %s sekund" % (end_time1 - start_time1))
print("Twój klucz prywatny: ", d)
print("Twój klucz publiczny: ", Q)
print("")

# Podpisywanie wiadomości
message = input("Wpisz wiadomość do podpisania: ").strip()
while len(message) == 0:
    message = input("Wpisz wiadomość do podpisania: ").strip()
start_time2 = time.time()
signature = sign_message(message, d, G)
end_time2 = time.time()
print("")
print("Czas wykonania podpisu: %s sekund" % (end_time2 - start_time2))
print("Podpis wiadomości (r, s):", signature)

# Weryfikacja podpisu
start_time3 = time.time()
is_valid = verify_signature(Q, message, signature)
end_time3 = time.time()
print("Czas wykonania weryfikacji: %s sekund" % (end_time3 - start_time3))
print("Czy podpis jest prawidłowy?", is_valid)
print("")
print("")

#-----

# Porównanie z gotową implementacją
start_time4 = time.time()
ecdsa_signature, vk = python_ecdsa_module_signature(cur, message)
end_time4 = time.time()
print(f"Czas podpisu dla gotowej implementacji: {end_time4 - start_time4} sekund")

start_time5 = time.time()
is_valid_ecdsa = python_ecdsa_module_verify(cur, ecdsa_signature, message, vk)
end_time5 = time.time()
print(f"Czas weryfikacji dla gotowej implementacji: {end_time5 - start_time5} sekund")
print("Czy podpis jest prawidłowy dla gotowej implementacji?", is_valid_ecdsa)
```

4. Instrukcja obsługi

Uwaga: Przed przeczytaniem tego rozdziału zapoznaj się z rozdziałem 3. „Opis aplikacji”.

1) Generowanie podpisu wiadomości

```
Wybierz krzywą eliptyczną na jakiej chcesz dokonać podpisu:  
1 - P-224, 2 - P-256, 3 - P-384, 4 - P-521  
Podaj cyfrę: 
```

Po uruchomieniu głównej komórki aplikacji, w polu poniżej kodu, aplikacja rozpocznie komunikację z użytkownikiem. Na początek należy wybrać krzywą eliptyczną, która zostanie wykorzystana do utworzenia podpisu wiadomości. Celem dokonania wyboru, naciśnij na klawiaturze przycisk z cyfrą przypisaną do wybranej krzywej. Aby zatwierdzić wybraną liczbę naciśnij przycisk Enter.

```
Podaj cyfrę: 3  
Wybrałeś krzywą P-384.  
Czas generowania kluczy: 0.03243708610534668 sekund  
Twój klucz prywatny: 2237120886812796321095644638102051055653229003219480693369  
Twój klucz publiczny: (28074807746669409737206347829432318514038831489328521089  
Wpisz wiadomość do podpisania: 
```

Po dokonaniu wyboru krzywej eliptycznej (w tym przypadku: P-384) aplikacja wyświetli wygenerowany na wybranej krzywej klucz prywatny i publiczny (klucz publiczny składa się z dwóch wartości oddzielonych w nawiasie przecinkiem) oraz pole do wpisania wiadomości, którą chcesz podpisać. Po wpisaniu wiadomości naciśnij przycisk Enter.

```
Wpisz wiadomość do podpisania: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna al  
Czas wykonania podpisu: 0.03416872024536133 sekund  
Podpis wiadomości (r, s): (27917729041312080546006921028349556830393653807848237230903358690215508791469854388719629356807235799759577339487251, 2777  
Czas wykonania weryfikacji: 0.061449289321899414 sekund  
Czy podpis jest prawidłowy? True
```

Po wpisaniu wiadomości (w tym przypadku: akapit *Lorem ipsum*) wyświetlony zostanie wygenerowany podpis wiadomości składający się z wartości *r* oraz *s*. Obie wartości są konieczne do późniejszej prawidłowej weryfikacji podpisu, dlatego warto je zapisać! Domyślnie od razu następuje także weryfikacja podpisu – *True* jeżeli jest prawidłowy dla danej wiadomości, *False* jeżeli nieprawidłowy.

```
Czas podpisu dla gotowej implementacji: 0.003985881805419922 sekund  
Czas weryfikacji dla gotowej implementacji: 0.010404825210571289 sekund  
Czy podpis jest prawidłowy dla gotowej implementacji? True
```

Również domyślnie, poniżej wygenerowanego podpisu wiadomości, pojawiają się informacje nt. czasu wykonywania podpisu i jego weryfikacji przy użyciu gotowej biblioteki *ecdsa*. Te informacje można swobodnie pominąć w trakcie używania aplikacji.

2) Weryfikacja podpisu wiadomości

```
# Tylko weryfikacja podpisu wiadomości
# Pamiętaj o zamianie danych na własne!
Q = (493485494713930204926715936556911661911800419007479343477787809223253976451198534003815696140689809864803182332653, 42868561012
message = "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua"
signature = (27917729041312080546006921028349556830393653807848237230903358690215508791469854388719629356807235799759577339487251, 2
is_valid = verify_signature(Q, message, signature)

print("Czy podpis jest prawidłowy?", is_valid)

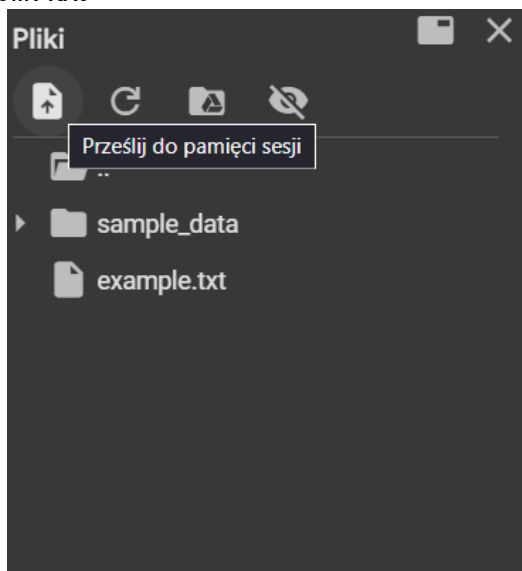
Czy podpis jest prawidłowy? True
```

Jeżeli w ramach aplikacji interesuje Cię jedynie weryfikacja podpisu wiadomości ECDSA, to możesz uruchomić specjalnie do tego przeznaczoną komórkę, która została umieszczona tuż pod główną komórką kodu. Domyślnie w komórce zdefiniowane zostały już dane (klucz publiczny pod zmienną *Q*, wiadomość pod zmienną *message* oraz podpis pod zmienną *signature*), które będziesz musiał zmienić w celu weryfikacji podpisu ECDSA, który posiadasz.

3) Podpis wiadomości z pliku .txt

Dodatkową funkcjonalnością aplikacji jest możliwość wygenerowania podpisu dla danych tekstowych zawartych w pliku .txt.

- a) Załaduj plik .txt do środowiska w Google Colab: kliknij w ikonkę z folderem, znajdującą się na pasku z lewej strony. Otworzy się zakładka „Pliki”. Kliknij ikonkę kartki z strzałką i prześlij do Colab’a swój plik .txt



- b) „Odkomentuj” funkcję `load_text_file()` znajdującą się w komórce z funkcjami pomocniczymi. Załaduj ponownie komórkę z funkcjami pomocniczymi.

```
# Funkcja umożliwiająca załadowanie tekstu jako wiadomości z pliku .txt
def load_text_file(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            text_content = file.read()
        return text_content
    except Exception as e:
        print(f"Wystąpił błąd podczas otwierania pliku: {e}")
        return None

# Przykład użycia
file_path = "example.txt" # Podaj ścieżkę do pliku
message = load_text_file(file_path)
```

- c) „Zakomentuj”, tak jak na rysunku poniżej, fragment kodu (3 linijki) w głównej komórce aplikacji

```
# GŁÓWNA KOMÓRKA APLIKACJI
# Wybór krzywej
p, a, b, G, n, cur = choose_curve()

# Generowanie kluczy
start_time1 = time.time()
d, Q = generate_keys()
end_time1 = time.time()
print("Czas generowania kluczy: %s sekund" % (end_time1 - start_time1))
print("Twój klucz prywatny: ", d)
print("Twój klucz publiczny: ", Q)
print("")

# Podpisywanie wiadomości
# message = input("Wpisz wiadomość do podpisania: ").strip()
# while len(message) == 0:
# message = input("Wpisz wiadomość do podpisania: ").strip()
start_time2 = time.time()
signature = sign_message(message, d, G)
end_time2 = time.time()
print("")
print("Czas wykonania podpisu: %s sekund" % (end_time2 - start_time2))
print("Podpis wiadomości (r, s):", signature)
```

- d) Uruchom główną komórkę aplikacji... i gotowe!

Aby powrócić do domyślnego sposobu podpisywania wiadomości, „zakomentuj” z powrotem funkcję `load_text_file()`, załaduj ponownie komórkę zawierającą tę funkcję oraz „odkomentuj” linijki kodu w głównej komórce aplikacji.